

Rapport de projet informatique

Rapport de projet informatique

Projet réalisé du début du deuxième semestre au 23 mai 2026

Membres du groupe

Boumghar Yenni

Lien GitHub :

<https://github.com/Yenox499/Yenni.B.git>

Informations supplémentaires

N'ayant pas trouvé de personne sérieuse pour travailler avec moi, j'ai décidé de faire ce projet tout seul. Ainsi, il y a un seul membre dans le groupe car ceux qui devaient travailler dessus avec moi et qui ont approuvé le projet ne travaillaient pas et n'avaient pas l'air de vouloir faire la moindre tâche et ralentissaient plus qu'autre chose la réalisation de ce projet. Il n'était pas du tout intéressé. Étant très intéressé par ce projet, je n'ai pas eu d'autre choix que de le faire tout seul.

Table des matières

1	Introduction	4
2	Environnement de travail	5
3	Description du projet et objectifs	6
3.1	Qu'est-ce que l'outil et quelle en sont ces objectif	6
3.2	Comment ça marche ?	6
3.3	Ce que l'outil couvre et ne couvre pas	7
4	Bibliothèques, Outils et technologies	8
5	Travail réalisé	10
6	Difficultés rencontrées	11
7	Bilan	13
7.1	Conclusion	13
7.2	Perspectives	13
8	Bibliographie	14
9	Webographie	15
10	Annexes	16
A	Cahier des charges	16
B	Exemple d'exécution du projet	18
C	Manuel utilisateur	21

1 Introduction

Le marché des PC assemblés souffre d'un manque flagrant de transparence qui pénalise les consommateurs non avertis. En effet, lorsqu'un particulier consulte la fiche d'un PC sur un site comme Amazon, Cdiscount ou en magasin, il se retrouve face à des descriptions souvent incomplètes ou volontairement vagues : les composants sont mentionnés de manière superficielle, les marques des pièces internes sont rarement précisées, et les références exactes sont quasi absentes. Cette opacité n'est pas anodine : elle rend toute comparaison sérieuse pratiquement impossible pour un acheteur lambda, qui ne dispose ni des connaissances techniques ni des outils nécessaires pour décortiquer une configuration et en évaluer le juste prix. Ainsi, l'objectif général de ce projet, c'est de rendre accessible à tous une information qui était jusqu'ici réservée aux experts.

Je vais vous présenter un projet qui me tient à cœur et qui a son importance pour la société actuelle. Dans ma jeunesse, j'étais fan de PC gamer, je sais les monter et je m'y connais beaucoup dans le domaine du hardware.

Lorsque je m'intéressai énormément à ce marché, je me suis rendu compte que beaucoup d'entreprises en profitaient sur les prix de leurs PC déjà montés en grande surface ou sur des sites internet très connus, car il est difficile pour des particuliers qui ne s'y connaissent pas dans ce domaine de s'y retrouver. J'ai décidé de convier mon projet à cette cause. On va d'ailleurs découvrir à travers l'outil que j'ai développé les marges assez impressionnantes que peuvent se faire les entreprises sur certaines ventes de PC sur internet. L'outil se nomme "B.pc".

2 Environnement de travail

Pour réaliser ce projet, j'ai utilisé ma machine, un Mac avec puce Apple intégrée et 16 Go de RAM, permettant à la fois de supporter les ressources que demandent les logiciels d'intelligence artificielle et de gérer de multiples tâches sans bug. J'ai donc utilisé ma machine pour créer de A à Z ce projet.

J'ai utilisé Visual Studio Code comme éditeur de code principal, dans lequel figure l'ensemble de mes fichiers de code, notamment les fonctions Python du backend. Pour les fichiers HTML, CSS et JavaScript, j'ai utilisé Phoenix Code, qui me permet de visualiser en temps réel les modifications apportées au design de la plateforme directement dans l'éditeur, sans avoir à rafraîchir manuellement le navigateur.

Le projet s'exécute entièrement en local sur ma machine, sans nécessiter de déploiement sur un serveur distant. Pour lancer l'application, un script de démarrage a été mis en place, permettant de lancer le serveur en un simple double-clic, sans avoir à ouvrir un terminal ou à saisir la moindre commande. Ce choix a été fait dans un souci de simplicité et d'accessibilité. Une fois lancé, le serveur Python tourne localement et l'interface de l'application est accessible directement depuis un navigateur web à l'adresse localhost :8000.

Enfin, j'ai utilisé comme langage de programmation principal Python pour coder l'ensemble de mes fonctions et autres codes, HTML, CSS et JS pour l'interface web et SQLite pour la base de données.

3 Description du projet et objectifs

3.1 Qu'est-ce que l'outil et quelle en sont ces objectif

Le projet B.pc poursuit un objectif principal clair et ambitieux : permettre à n'importe quel particulier de savoir en quelques secondes s'il est plus avantageux d'acheter un PC déjà assemblé en magasin ou sur internet ou s'il est plus avantageux de le monter soi-même en achetant les composants séparément.

L'objectif de l'outil est de comparer le prix d'un PC gaming ou bureautique vendu assemblé sur internet avec le coût total de ses composants achetés séparément, en recherchant pour chaque composant le tarif le plus bas disponible sur le marché au moment de l'analyse.

Ainsi, grâce à la déduction de ces deux prix, on peut faire la soustraction du prix du PC acheté complet avec le prix de la somme de ces composants recherchés sur internet et en déduire s'il serait plus rentable pour le particulier de monter le PC tout seul en achetant directement les pièces séparément ou d'acheter le PC directement monté.

Je précise aussi que le montage d'un PC est quelque chose d'ouvert à tous, une simple vidéo sur internet permet largement de pouvoir monter soi-même son PC même si on ne s'y connaît pas du tout (pertinent pour comprendre la logique du projet).

Concrètement, l'outil compare le prix d'un PC vendu en version complète sur internet avec la somme des prix de ses composants recherchés individuellement au meilleur tarif disponible sur le marché, et en déduit automatiquement l'économie potentielle qui pourrait être réalisée par le particulier souhaitant acheter un PC. L'outil promet une rapidité de comparaison raisonnable ainsi qu'une approximation du tarif que pourrait économiser le client.

L'outil a deux objectifs principaux. Le premier est celui de l'accessibilité : l'outil doit pouvoir être utilisé par une personne n'ayant aucune connaissance en informatique. Pas de jargon technique, pas d'étapes complexes, l'utilisateur n'a qu'à coller le lien d'un PC trouvé sur internet et l'outil fait le reste.

Le deuxième est à la fois de faire économiser les gens mais aussi de faire comprendre aux particuliers et aux entreprises présentes dans ce secteur que quelquefois les tarifs appliqués aux PC vendus sur internet sont beaucoup trop excessifs à l'égard des personnes qui ignorent le véritable prix et qui plongent très souvent dans l'arnaque.

Remarque : pour les biens de ce projet, seuls les PC gamer ou PC bureautique du site LDLC peuvent être comparés. La programmation des algorithmes, fonctions et autres permettant de faire fonctionner l'outil pour des fiches produit de PC LDLC sont globalement les mêmes pour d'autres sites (Amazon, Cdiscount, Pccomponent), à quelques différences près. Ainsi, seul le site LDLC (très grand revendeur de PC en France) sera pris comme exemple. L'outil peut bien entendu être par la suite étendu à n'importe quel site de revendeurs de PC.

3.2 Comment ça marche ?

Le fonctionnement de B.pc repose sur un pipeline automatisé en plusieurs étapes qui s'enchaînent de manière transparente pour l'utilisateur. Voici le détail de chaque étape :

Étape 1 — Saisie de l'URL. Tout commence par une action simple : l'utilisateur colle

dans l'interface de B.pc le lien d'un PC trouvé sur un site marchand (le site LDLC sera l'exemple de ce projet). C'est la seule manipulation demandée à l'utilisateur. À partir de ce moment, l'outil prend le relais et effectue l'intégralité de l'analyse de manière automatique.

Étape 2 — Scraping de la page produit. L'outil visite automatiquement la page du PC en question et en extrait toutes les informations pertinentes : nom du produit, prix de vente, description technique, et surtout la liste des composants mentionnés (processeur, carte graphique, mémoire vive, stockage, carte mère, alimentation, boîtier).

Étape 3 — Analyse par intelligence artificielle. Les données brutes extraites sont ensuite transmises à un module d'analyse basé sur l'intelligence artificielle. Ce module identifie et structure les composants détectés, en les normalisant pour les rendre exploitables pour la recherche de prix.

Étape 4 — Recherche du meilleur prix par composant. Pour chaque composant identifié, l'outil interroge des API de recherche de prix afin de trouver le tarif le plus bas disponible sur le marché au moment de l'analyse. Cette recherche est effectuée composant par composant, à la référence près, pour garantir la meilleure approximation possible du coût de montage.

Étape 5 — Comparaison et verdict final. Une fois tous les prix collectés, l'outil calcule la somme totale des composants achetés séparément et la compare au prix du PC assemblé. La différence obtenue permet de déduire automatiquement un verdict clair : est-il plus rentable d'acheter le PC monté, ou de l'assembler soi-même ? Ce verdict est affiché à l'utilisateur de manière simple et lisible, accompagné du montant de l'économie potentielle réalisable.

3.3 Ce que l'outil couvre et ne couvre pas

Tout d'abord L'outil couvre deux grandes catégories de machines : les PC gaming, destinés aux joueurs souhaitant une configuration performante pour les jeux vidéo, et les PC bureautiques, destinés à un usage plus classique tel que le travail, les études ou la navigation internet.

Il est aussi important de bien comprendre ce que l'outil calcule et ce qu'il ne calcule pas. Le projet fournit une approximation de l'économie potentielle réalisable en montant soi-même son PC, et non un calcul exact au centime près. Cette approximation repose sur les meilleurs prix trouvés au moment de l'analyse pour chaque composant, et peut donc varier légèrement dans le temps en fonction des fluctuations du marché. Elle reste néanmoins suffisamment fiable pour permettre à l'utilisateur de prendre une décision d'achat éclairée.

4 Bibliothèques, Outils et technologies

Cette section présente l'ensemble des bibliothèques, outils et technologies qui ont permis de concevoir et de faire fonctionner le projet. Afin d'en faciliter la lecture, cette section est organisée en quatre parties distinctes : le backend, qui regroupe tout ce qui se passe côté serveur ; le frontend, qui regroupe les technologies utilisées pour construire l'interface utilisateur ; le scraping et la recherche de prix, qui décrivent les outils permettant d'extraire les données des fiches produits et de trouver les meilleurs prix sur le marché ; les API externes, qui présentent les services tiers intégrés au projet.

Le backend de mon outil est développé en Python. Pour faire tourner le serveur, j'ai utilisé FastAPI, un framework Python spécialement conçu pour la création d'API web. FastAPI m'a permis de faire le lien entre l'interface que voit l'utilisateur dans son navigateur et tout ce qui se passe coté serveur. Concrètement : il coordonne chacune des étapes dans l'ordre : il envoie l'URL au module de scraping, il transmet les données récupérées à l'API d'affinage des références, il envoie chaque composant à l'API de recherche de prix, il calcule la différence et prépare le résultat final et c'est aussi lui qui, lorsque j'ouvre la plateforme locale dans le navigateur, affiche l'interface HTML/CSS/JS de mon outil. De plus, j'ai aussi utilisé Uvicorn, un serveur qui travaille directement avec FastAPI et Pydantic, une bibliothèque de validation de données . Elle s'assure que les données qui circulent dans l'outil ont le bon format.

J'ai utiliser, HTML, CSS et JS. Pour avoir un design de qualité, j'ai booster mes fichier grâce a claude design qui m'a généré des fichier html, CSS et JS selon mes gout. La générations des fichiers n'étant pas parfaite, j'ai moi même modifier quelque ligne de code dans les fichiers mais globalement ce que l'outil IA est capable de faire est tout simplement impressionnant.

L'extraction et la recherche de données reposent sur trois outils complémentaires qui interviennent successivement dans le pipeline.

J'ai utilisé Playwright pour la première extraction du HTML de la page, une bibliothèque installée localement. C'est lui qui se charge du scraping des pages produits LDLC, les pages LDLC se chargent avec JavaScript, donc on ne peut pas juste "télécharger" la page comme un fichier, il faut qu'un vrai navigateur l'ouvre pour voir son contenu réel. Playwright attend que l'intégralité du contenu soit rendue par le navigateur avant d'en extraire le HTML brut, garantissant ainsi une récupération complète et fiable des informations.

Vient ensuite l'outil BeautifulSoup4, une bibliothèque Python qui sert à lire et extraire du contenu HTML. Une fois le HTML brut récupéré par Playwright, BeautifulSoup4 se charge de le lire et d'en extraire uniquement les informations pertinentes que je souhaiter : le nom du PC, son prix, et la description technique contenant les composants (8) dont j'avais besoin. BeautifulSoup4 m'a ainsi transformé un bloc de code HTML brut et difficile à exploiter en données propres et structurées, prêtes à être transmises à l'intelligence artificielle pour analyse.

SertAPI et GroqAPI (Llama 3.3) interviennent en dernière étape de la recherche de données. Ces deux outils sont complémentaires et indissociables dans le pipeline du projet. Llama 3.3 reçoit ce texte extrait par BeautifulSoup4 et le transforme en réf-

rences précises et en requêtes de recherche exploitables : "Intel Core i5 3,5 GHz"(peu clair et tres vague) devient "Intel Core i5-13400F prix France" . SertAPI va ensuite recevoir ces requêtes précises générées par Llama 3.3, les tape dans Google Shopping et retourne le meilleur prix trouvé pour chaque composant.

Ces quatre outils forment une chaîne cohérente et efficace : Playwright récupère, BeautifulSoup4 structure, Groq/Llama 3.3 affine intelligemment et SertAPI valorise à travers google shopping .

Je rajoute quelque précision technique sur les API qui ont été utilisé pour ce projet. J'ai utilisé Llama 3.3 accessible via Groq. Llama 3.3 est une API rapide, efficace et gratuit et qui m'a permis d'analyser les composants et de générer les requêtes de recherche. Utilisable via une clé API, je l'ai surtout choisie pour sa vitesse de recherche exceptionnelle. Très simple d'utilisation, c'est lui qui analyse les composants scrappés et les enrichit (raffine les noms génériques, estime les composants manquants). Il identifie les références exactes des composants mal nommés, devine et complète les pièces manquantes et vérifie la compatibilité entre les composants refusant par exemple d'associer un processeur Intel LGA1700 avec une carte mère socket AM5 (incompatible).

Il génère enfin des requêtes de recherche précises et optimisées qui sont transmises à SerpAP. En résumé, il comprend, devine, raisonne et vérifie intelligemment.

SertAPI est une autre API que j'ai utilisé. Elle m'a surtout permis d'interroger Google Shopping pour trouver le meilleur prix par composant. L'avantage est que SertAPI est un service qui agit comme un intermédiaire entre le code et Google et qui contourne les protections anti-bot de Google qui empêcheraient l'outil de faire des recherches automatisées directement. Il supporte Google Shopping, Google Search... Les résultats retournés par SertAPI sont en JSON, un format structuré facile à lire et à exploiter directement en Python, ce qui s'intègre parfaitement avec FastAPI et Pydantic.

Ainsi, en résumé à tout cela, L'intelligence artificielle constitue la pièce maîtresse de mon projet, celle sans laquelle l'ensemble du pipeline s'effondrerait. Elle repose sur deux composantes distinctes mais indissociables, SertAPI et Llama 3.3.

5 Travail réalisé

Au terme du développement, plusieurs fonctionnalités majeures ont été intégralement réalisées et sont pleinement opérationnelles dans B.pc. Voici la liste des tâches qui devaient être initialement effectuées :

- Scraping LDLC : extraction automatique des composants depuis une URL produit
- Enrichissement IA : raffine les noms génériques, estime les composants manquants
- Recherche de prix Google Shopping : rechercher les composants les moins chers
- Calcul du verdict : BON DEAL / BIG DEAL / DEAL OF THE LIFE / PAS DE DEAL
- Une interface web
- Support multi-marchands : Amazon, Cdiscount, TopAchat, Fnac, Boulanger...

Le pipeline complet représente la réalisation technique la plus ambitieuse du projet. L'enchaînement Playwright - BeautifulSoup4 - Llama 3.3 - SerpAPI a été intégralement développé et connecté : Playwright scrape la page produit LDLC en mode headless, BeautifulSoup4 parse et structure le HTML récupéré, Llama 3.3 analyse les composants et génère des requêtes de recherche précises, et SerpAPI interroge Google Shopping pour trouver le meilleur prix disponible pour chaque pièce. Chaque brique communique correctement avec la suivante, et le pipeline produit en sortie une estimation chiffrée de l'économie potentielle réalisable.

Les prix plancher et plafond par composant constituent la dernière fonctionnalité réalisée et ajoutée au fil du projet, et est l'une des plus importantes pour la fiabilité de l'outil. Pour chaque catégorie de composant, processeur, carte graphique, mémoire vive, stockage, etc., des seuils minimum et maximum de prix ont été définis et intégrés dans le pipeline. Ces seuils permettent de filtrer automatiquement les résultats aberrants retournés par SerpAPI : un processeur affiché à 5€ ou à 5 000€ sera automatiquement écarté, garantissant ainsi que l'estimation finale repose sur des données réalistes et cohérentes avec le marché actuel.

Une seule fonctionnalité n'a pas été réalisée et c'est la possibilité de rentrer un lien autre que ceux provenant du site LDLC. La raison de ce choix est que rendre extensible la plateforme à d'autres sites demanderait énormément de temps et il n'y avait pas grand intérêt puisque le but du projet était de sortir un outil fonctionnel, le site n'a été qu'un exemple ici pour prouver que l'outil fonctionne bien.

Après de nombreux tests effectués, de nombreuses améliorations, la plateforme génère une approximation de l'économie que pourrait réaliser un particulier sur un PC avec une marge d'erreur de 5 à 15 % suivant le PC et les composants qu'il comporte.

6 Difficultés rencontrées

Le scraping et les fiches produits ont été la première difficulté que j’ai rencontrée. Que ce soit pour le site LDLC ou pour d’autres sites, le scraping est l’étape la plus complexe car les composants n’apparaissent pas clairement et concrètement tout le temps, les références exactes n’apparaissent pas, ainsi cela rend d’avance très compliquée l’étape du scraping.

Si on prend comme exemple le site LDLC (très souvent on retrouvera le même problème sur presque tous les sites de revendeurs de PC comme Amazon. . .), la première et principale difficulté technique rencontrée dans ce projet a été l’opacité de ces fiches produits. Dès les premières phases de test, il est apparu que les informations disponibles sur les pages produits du site étaient largement insuffisantes pour permettre une recherche de prix directe et fiable sur Google Shopping. Le problème se manifestait de trois manières distinctes.

- Premièrement, les composants étaient décrits de manière extrêmement vague, sans référence exploitable : un processeur était mentionné comme "Intel Core i5 3.5 GHz" au lieu de sa référence exacte "Intel Core i5-13400F", une carte graphique apparaissait simplement comme "NVIDIA GeForce RTX 4070" sans précision sur la mémoire vidéo, et un boîtier était décrit uniquement comme "Moyen Tour" sans aucun nom de marque ni de modèle.
- Deuxièmement, certains composants étaient tout simplement absents de la fiche produit, c’est notamment le cas du système de refroidissement du processeur, qui n’est que rarement mentionné sur les pages LDLC alors qu’il représente une pièce indispensable à toute configuration, ou encore l’alimentation avec la référence exacte.
- Troisièmement, les informations fournies étaient parfois incohérentes ou incomplètes, rendant toute tentative d’identification automatique des composants particulièrement hasardeuse sans une analyse intelligente des données.

Sans références exactes, SerpAPI se retrouvait dans l’incapacité de trouver les bons produits sur Google Shopping. Une recherche avec "Intel Core i5 3,5 GHz prix" retourne des dizaines de résultats non pertinents, tandis qu’une recherche avec "Intel Core i5-13400F prix France" cible immédiatement le bon produit au bon prix. De même, un composant absent de la fiche faussait complètement l’estimation du coût total de montage.

C’est précisément pour résoudre cette difficulté que Llama 3.3 a été intégré au pipeline. Le recours à un modèle de langage puissant s’est imposé comme la seule solution viable. Mais malgré l’ajout de Llama 3.3, les informations hasardeuses et manquantes ont quand même posé difficulté à l’agent IA, qui a dû être affiné à de multiples reprises pour corriger énormément de résultats incohérents.

Il y a eu aussi le problème de l’hallucination de Llama 3.3.

L’intégration d’un modèle de langage comme Llama 3.3 apporte une puissance d’analyse considérable, mais elle introduit également un risque inhérent : le langage génère des informations qui semblent plausibles et cohérentes en apparence, mais qui sont en réalité sont fausses ou inexistantes.

Llama 3.3 pouvait par exemple inventer une référence de processeur qui n’existe pas sur le marché, générant ainsi une requête de recherche SerpAPI qui ne retournerait aucun résultat valide. Il pouvait également proposer un composant de refroidissement

incompatible avec le socket du processeur identifié, ou suggérer une carte mère dont la référence exacte n'est commercialisée par aucun fabricant. Dans tous ces cas, l'estimation finale produite par B. pc serait faussée, voire complètement inutilisable, sans que l'utilisateur en soit averti.

Ce risque est problématique car les hallucinations de Llama 3.3 sont difficiles à détecter : une référence inventée comme "Intel Core i5-13402F" ressemble à une vraie référence comme Intel Core i5-13400F", et seule une vérification sur le marché permet de confirmer son existence réelle.

Plusieurs mécanismes ont été mis en place pour limiter ce risque :

- Premièrement, les planchers et plafonds de prix par composant : si SerpAPI ne trouve aucun résultat pour une référence inventée, ou retourne un prix complètement aberrant, le composant est automatiquement écarté du calcul.

Deuxièmement, les vérifications de compatibilité socket intégrées dans les instructions données à Llama 3.3 réduisent significativement le risque de proposer des associations de composants incohérentes.

- Troisièmement, le modèle est interrogé avec des prompts très précis et contraints, qui limitent sa marge de liberté et l'orientent vers des références réelles plutôt que vers des inventions plausibles.

Bien que l'IA soit très utile, aucun système ne peut garantir à 100 % l'absence d'hallucinations dans les résultats produits par la plateforme. C'est précisément pourquoi l'outil se présente comme une approximation et non comme une vérité absolue.

Une autre difficulté rencontrée a été la fiabilité des prix trouvés via SerpAPI.

L'un des défis a été de garantir la fiabilité des prix retournés par SerpAPI. Mais il y eut plusieurs problèmes rencontrés :

- Le problème des revendeurs peu fiables a été le premier obstacle identifié. Google Shopping agrège les offres de centaines de revendeurs, parmi lesquels figurent des marketplaces de qualité très variable : revendeurs inconnus, boutiques en liquidation, vendeurs tiers sur des plateformes comme Amazon ou Cdiscount dont les prix ne correspondent pas toujours à la réalité du marché.

- Le problème des prix obsolètes constituait un deuxième problème . Google Shopping indexe parfois des offres qui ne sont plus disponibles, des stocks épuisés affichés à des prix promotionnels dépassés, ou des fiches produits dont le prix n'a pas été mis à jour depuis plusieurs semaines.

- Le problème des résultats hors sujet. Une requête comme "Intel Core i5-13400F prix France" peut parfois retourner des résultats concernant des produits reconditionnés ou encore des versions OEM destinées aux constructeurs.

Pour faire face à ces trois problèmes, une solution a été mise en place. Pour chaque catégorie de composant, des seuils minimum et maximum ont été définis en se basant sur la réalité du marché actuel. Tout résultat SerpAPI sortant de ces fourchettes est automatiquement filtré et écarté du calcul, forçant l'outil à ne retenir que les prix cohérents avec la réalité du marché. Cette solution ne garantit pas une fiabilité absolue, mais elle réduit considérablement le risque de voir l'estimation finale faussée par un résultat aberrant.

7 Bilan

7.1 Conclusion

Il a été question pour moi dans ce projet de découvrir ce qui était possible de réaliser grâce à l'intelligence artificielle et c'est vraiment bluffant.

À partir d'une instruction de départ et de quelques étapes mises en place avec mon agent IA Claude Code, j'ai réussi à développer un outil fonctionnel et fiable. La part de responsabilité du développeur reste quand même conséquente car sans lui pour tout coordonner, rien ne fonctionnerait correctement. Malgré quelques bugs et quelques difficultés, ce que l'intelligence artificielle est capable de faire est tout à fait remarquable. Ma stupéfaction a plutôt été tournée vers sa capacité à comprendre en détail ce que je lui demandais et à le faire en un temps record, entre 20 et 30 minutes pour certaines tâches. Ce qui prendrait pour un être humain plusieurs mois prend à l'agent IA une trentaine de minutes et ce qui est encore plus bluffant, c'est qu'elle retourne un résultat tout à fait juste et sans erreur.

J'ai aussi découvert à travers ce projet ce qu'était une API, c'est tout aussi remarquable et ça rejoint ce que j'ai évoqué en haut, une simplicité d'utilisation et une vitesse d'exécution assez dingue pour des tâches très complexes. Malgré de multiples difficultés, l'IA arrive quand même à s'en sortir.

J'ai découvert comment fonctionnait le backend et comment se déroulait la liaison entre tous les composants de la pipeline, le frontend et autres. J'ai aussi découvert des outils Python, de multiples bibliothèques et comment tous ça fonctionnait.

Ainsi, c'est un projet qui m'a beaucoup plu, qui m'a énormément appris et j'ai en plus en ma possession un outil tout à fait fonctionnel dont je vais pouvoir continuer le développement.

7.2 Perspectives

Les perspectives sont nombreuses et innovantes !

Les perspectives à court terme :

- Correction de bugs et affinage des outils de manière à avoir une plateforme 100 % fiable, fluide et optimale.

Les perspectives à moyen terme, les évolutions fonctionnelles :

- Élargir à d'autres sites que LDLC : Amazon, Cdiscount, Fnac, Materiel.net... sont autant de sources potentielles avec des millions de PC référencés.
- Ajouter un historique des analyses, permettre à l'utilisateur de sauvegarder et comparer plusieurs PC analysés.
- Générer une liste de courses, un récapitulatif cliquable de tous les composants avec leur meilleur prix et leur lien d'achat.

Les perspectives à long terme, ma vision ambitieuse :

- Déployer l'outil en ligne, passer d'une plateforme local à un vrai site accessible à tous, sans installation.
- Couvrir d'autres catégories, étendre le concept aux smartphones, téléviseurs, ou tout autre produit électronique composé de pièces identifiables.
- Créer une application mobile, rendre l'outil accessible directement depuis un télé-

phone, en scannant un QR code ou en collant un lien.

On pourrait même y ajouter une perspectives sociétales :

- Partenariats avec des associations de consommateurs, UFC-Que Choisir, 60 Millions de Consommateurs, pour donner plus de visibilité à l'outil.
- Sensibilisation au montage PC, intégrer des tutoriels ou des liens vers des ressources pour encourager les utilisateurs à franchir le pas du montage.

8 Bibliographie

Aucune source papier n'a été utilisée pour ce projet.

9 Webographie

[Langage] docs.python.org/3.13/

[Langage] developer.mozilla.org/fr/

[Backend] fastapi.tiangolo.com

[Backend] uvicorn.org

[Backend] docs.pydantic.dev

[Backend] pypi.org/project/python-dotenv

[Scraping] playwright.dev/python/docs/intro

[Scraping] crummy.com/software/BeautifulSoup/bs4/doc/

[Scraping] docs.python-requests.org

[IA] console.groq.com/docs/overview

[IA] huggingface.co/meta-llama/Llama-3.3-70B-Instruct

[IA] huggingface.co/meta-llama/Meta-Llama-3.1-8B-Instruct

[Prix] serpapi.com/google-shopping-api

[Prix] serpapi.com/dashboard

[Données] ldlc.com

[Outils] code.visualstudio.com

[Outils] pypi.org

[Outils] git-scm.com/doc

[Référence] developer.mozilla.org/fr/docs/Web/API/Fetch_API

[Référence] developer.mozilla.org/fr/docs/Web/CSS/CSS_grid_layout

[IA] chatgpt.com

[IA] copilot.microsoft.com

[IA] claude.ai

10 Annexes

Annexe A : Cahier des charges

Projet : B.pc - Comparateur de prix PC - Établissement : Université Paris Nanterre,
Licence MIASHS première année Date de début : Semestre 2

1. Objectif général

Développer une plateforme locale permettant à n'importe quel utilisateur, quel que soit son niveau de connaissance en informatique, de déterminer s'il est plus avantageux financièrement d'acheter un PC assemblé ou de le monter soi-même en achetant les composants séparément.

2. Périmètre du projet

- Site couvert : LDLC
- Types de PC analysés : PC gaming et PC bureautiques
- Mode de fonctionnement : plateforme web locale accessible via navigateur à l'adresse "localhost : 8000"
- Utilisateurs cibles : grand public non initié, parents, étudiants, toute personne souhaitant acheter un PC sans s'y connaître.

3. Fonctionnalités attendues

Fonctionnalités principales :

- Permettre à l'utilisateur de soumettre l'URL d'un PC LDLC via une interface web.
- Scraper automatiquement la page produit pour en extraire les informations techniques
- Identifier et normaliser les composants du PC grâce à l'intelligence artificielle (Llama 3.3)
- Rechercher le meilleur prix disponible sur le marché pour chaque composant via Google Shopping.
- Calculer la différence entre le prix du PC assemblé et la somme des composants achetés séparément.
- Afficher un verdict clair et compréhensible indiquant si le montage soi-même est plus avantageux.

Fonctionnalités secondaires :

- Filtrer les prix aberrants grâce à des planchers et plafonds de prix par catégorie de composant
- Proposer les composants manquants non mentionnés dans la fiche produit.
- Vérifier la compatibilité entre les composants identifiés.
- Permettre le lancement de la plateforme en un simple double-clic sans manipulation technique

4. Contraintes techniques

- Langage principal : Python

- Framework serveur : FastAPI + Uvicorn
- Scraping : Playwright (Chromium headless) + BeautifulSoup4
- Intelligence artificielle : Llama 3.3 70B via Groq API
- Recherche de prix : SerpAPI (Google Shopping)
- Frontend : HTML, CSS, JavaScript
- Environnement d'exécution : local, macOS
- Aucune installation requise pour l'utilisateur final.

5. Contraintes de design

- Interface en thème épuré, éclairé
- Palette de couleurs : orange et blanc crème
- Police : Instrument Arial
- Chat animé pour l'affichage progressif des résultats
- Expérience utilisateur accessible à tous, sans jargon technique

6. Contraintes de fiabilité

- L'outil fournit une approximation des économies réalisables, et non un calcul exact.
- Les prix affichés sont ceux disponibles au moment de l'analyse et peuvent varier.
- Les éléments hors périmètre (garantie, SAV, frais de livraison, temps de montage) ne sont pas pris en compte dans le calcul.

7. Livrables attendus

- plateforme web fonctionnelle accessible sur localhost :8000
- Script de lancement double-clic
- Interface utilisateur complète et fonctionnelle
- Pipeline complet Playwright - BeautifulSoup4 - Llama 3.3 - SerpAPI
- Rapport de projet complet
- Manuel utilisateur

Annexe B : Exemple d'exécution du projet

Exemple de l'analyse de la fiche produit suivante : <https://www.ldlc.com/fiche/PB00607532.html>

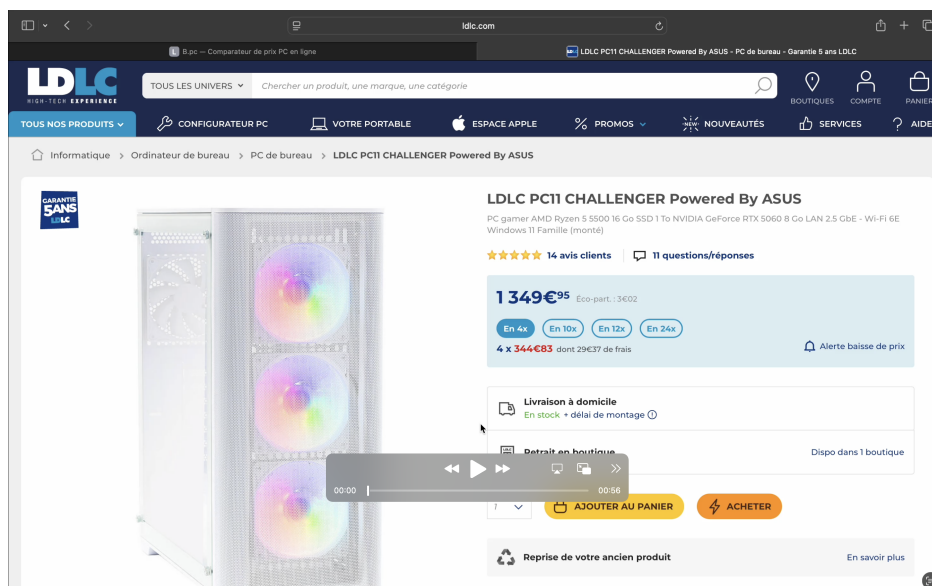


FIGURE 1 – Cliquez sur l'image pour voir la vidéo d'un exemple d'une analyse complète

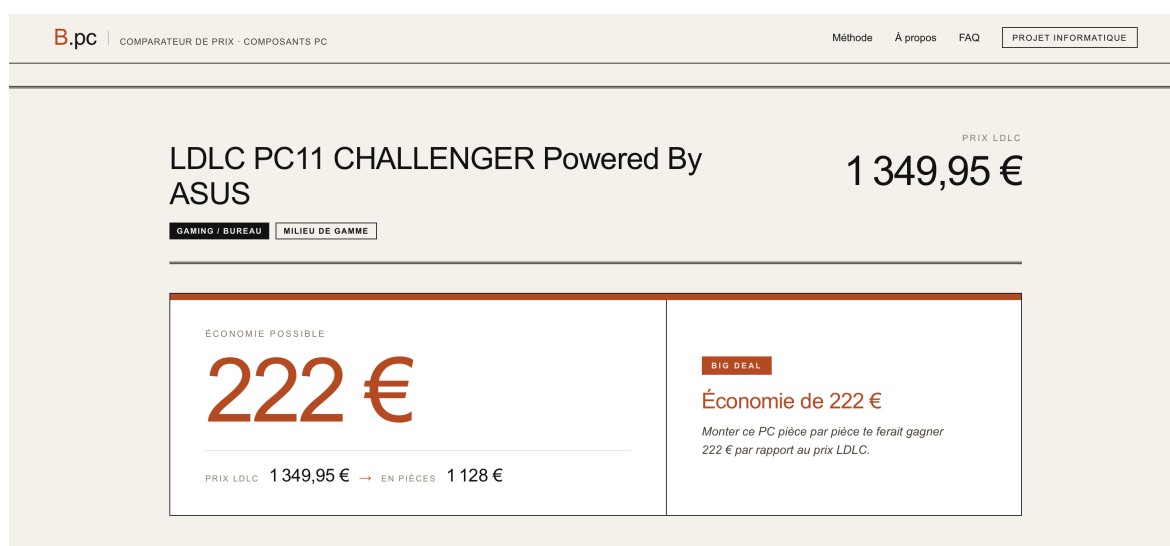


FIGURE 2 – Première visualisation de l'estimation des gains

Prix individuels · meilleure offre trouvée					1 128 €
COMPOSANT	RÉFÉRENCE	STATUT	PRIX	MARCHAND	LIEN
CPU	AMD Ryzen 5 5500 (3.6 GHz / 4.2 GHz)	TROUVÉ	85 €	Cdiscount	VOIR →
GPU	ASUS Dual GeForce RTX 5060 OC Edition 8GB	TROUVÉ	357 €	Amazon.fr - Seller	VOIR →
RAM	Corsair Vengeance 16Go DDR4 3200MHz	AFFINÉ	129 €	Cdiscount	VOIR →
SSD 1	SSD M.2 NVMe PCIe 4.0 x4 1 To	TROUVÉ	83 €	brico-phone	VOIR →
CARTE MÈRE	ASUS ROG STRIX B550-F GAMING (WI-FI) II	TROUVÉ	288 €	Dero.fr	VOIR →
BOÎTIER	Fox Spirit FG1 (Blanc)	TROUVÉ	35 €	Materiel.net	VOIR →
ALIMENTATION	650W Textorm TX650M+	TROUVÉ	60 €	LDLC	VOIR →
REFROIDISSEUR	Noctua NH-U12S	ESTIMÉ	90 €	Darty	VOIR →
TOTAL DES PIÈCES SÉPARÉES			1 128 €		

FIGURE 3 – Liste des composants du PC, chacun possédant un lien vers son prix le moins cher du marché

RECAPITULATIF

PC prémonté LDLC

1 349,95 €

Composants achetés séparément

1 128 €

Économie réalisée

222 € de moins

big deal

Économie de 222 €

Belle économie de 222.02 € sur ce PC prémonté !

⚠

Les composants marqués :

ESTIMÉ

 ont été identifiés par IA — leur référence exacte n'était pas disponible sur la fiche produit.

← NOUVELLE ANALYSE

FIGURE 4 – Résultat final de la comparaison

Voici l'interface principale de la plateforme :

B.pc | COMPARATEUR DE PRIX - COMPOSANTS PC

Méthode À propos FAQ PROJET INFORMATIQUE

Votre PC en ligne est-il vraiment rentable ?

Collez le lien d'un PC vendu en ligne. Notre moteur extrait chaque composant, interroge une base de prix indépendante, et chiffre *en euro* l'écart entre le tarif demandé et le coût réel des pièces.

Une méthode indépendante, sans partenariat marchand ni commission d'affiliation. Mise à jour des prix en temps réel.

① URL DU PRODUIT

<https://www.ldlc.com/fiche/PB00XXXXXX.html>

Collez l'URL d'un PC en ligne pour démarrer l'analyse

LANCER L'ANALYSE →

[Voir un exemple de résultat →](#)

8 COMPOSANTS ANALYSÉS

3 MOTEURS DE PRIX INDÉPENDANTS

IA ENRICHISSEMENT LLAMA 3.3

0€ GRATUIT - SANS COMPTE

FIGURE 5 – Accueil de la plateforme

Trois étapes, *une réponse chiffrée.*

01
Extraction
Scraping de la fiche produit — composants, prix et configuration détaillée.

02
Enrichissement
Identification IA des références manquantes et vérification de compatibilité.

03
Verdict
Comparaison du prix préremonté contre le coût réel des pièces. À l'euro près.

B.PC COMPARATEUR DE PC

ÉDITION N° 0042 · V0.4 BÉTA · FRANCE

FIGURE 6 – Description de la plateforme

Annexe C : Manuel utilisateur

Prérequis :

Avant de pouvoir utiliser B.pc, assurez-vous de disposer des éléments suivants :

- Un ordinateur sous macOS.
- Une connexion internet active (indispensable pour les appels aux API de recherche de prix et d’intelligence artificielle)
- Un navigateur web installé (Chrome, Firefox, Safari ou tout autre navigateur moderne)

Étape 1 — Lancer l’application

Ouvrez le dossier du projet B.pc sur votre ordinateur. À l’intérieur, vous trouverez un fichier nommé `lancer le site.command`. Double-cliquez dessus pour démarrer l’application. Une fenêtre de terminal s’ouvre brièvement et le serveur démarre automatiquement en arrière-plan. Cette opération ne prend que quelques secondes.

Important : ne fermez pas cette fenêtre de terminal pendant l’utilisation de B.pc — elle fait tourner le serveur. Si vous la fermez, l’application s’arrête.

Étape 2 — Ouvrir l’interface

Une fois le serveur démarré, ouvrez votre navigateur web et tapez l’adresse suivante dans la barre d’adresse : `http://localhost:8000`
Appuyez sur Entrée. L’interface de B.pc s’affiche.

Conseil : si la page ne s’affiche pas, attendez quelques secondes et rafraîchissez la page avec `Cmd+Shift+R`.

Étape 3 — Trouver un PC à analyser

Rendez-vous sur le site LDLC (www.ldlc.com) et choisissez un PC gaming ou bureautique que vous souhaitez analyser. Une fois sur la page du produit, copiez l’URL complète depuis la barre d’adresse de votre navigateur.

Exemple d’URL valide : `https://www.ldlc.com/fiche/PB00684983.html`

Étape 4 — Lancer l’analyse

Retournez sur l’interface de B.pc à l’adresse `localhost :8000`. Collez l’URL copiée dans le champ prévu à cet effet et validez. L’outil lance alors automatiquement le pipeline d’analyse complet :

1. Scraping de la page produit LDLC
2. Identification des composants par Llama 3.3
3. Recherche du meilleur prix pour chaque composant via SerpAPI
4. Calcul de la différence de prix

Les résultats s’affichent progressivement dans l’interface du chat animée au fur et à mesure de l’analyse. Cette étape peut prendre entre 30 secondes et 2 minutes selon la

complexité de la configuration analysée et la vitesse de votre connexion internet.

Étape 5 — Lire les résultats

Une fois l'analyse terminée, la plateforme affiche un récapitulatif clair comprenant :

- Le prix du PC assemblé tel qu'affiché sur LDLC
- La liste des composants identifiés avec le meilleur prix trouvé pour chacun
- Le coût total estimé du montage soi-même
- Le verdict final : est-il plus avantageux d'acheter le PC assemblé ou de le monter soi-même ?
- L'économie potentielle réalisable en montant le PC soi-même

Rappel important : les résultats fournis par B.pc sont une approximation et non un calcul exact. Les prix affichés correspondent aux meilleurs tarifs trouvés au moment de l'analyse et peuvent varier dans le temps. La garantie, le SAV, les frais de livraison et le temps de montage ne sont pas pris en compte dans le calcul.

Étape 6 — Arrêter l'application

Pour arrêter B.pc, fermez simplement la fenêtre de terminal qui s'est ouverte lors du lancement. Le serveur s'arrête automatiquement.